CSC 402 | Lab 4 Report

1. Introduction and Objectives

This lab explores the Android Activity entry point and launcher configuration, state modeling with Kotlin data classes, stateless Jetpack Compose screens, previews, and event-driven state updates. The in-lab Attendance application and the individual StudyPlanner application were implemented and tested.

2. Task 1 — Activity and Launcher

Q1.1: The activity android:name value is .MainActivity.

Q1.2: The launcher intent filter contains <action android:name="android.intent.action.MAIN" /> and <category android:name="android.intent.category.LAUNCHER" />.

Q1.3: Inside onCreate, the code calls super.onCreate(savedInstanceState), then enableEdgeToEdge(), then setContent { ... }.

Q1.4: After temporarily commenting out the LAUNCHER category, the Lab4 icon was not present in the app drawer.

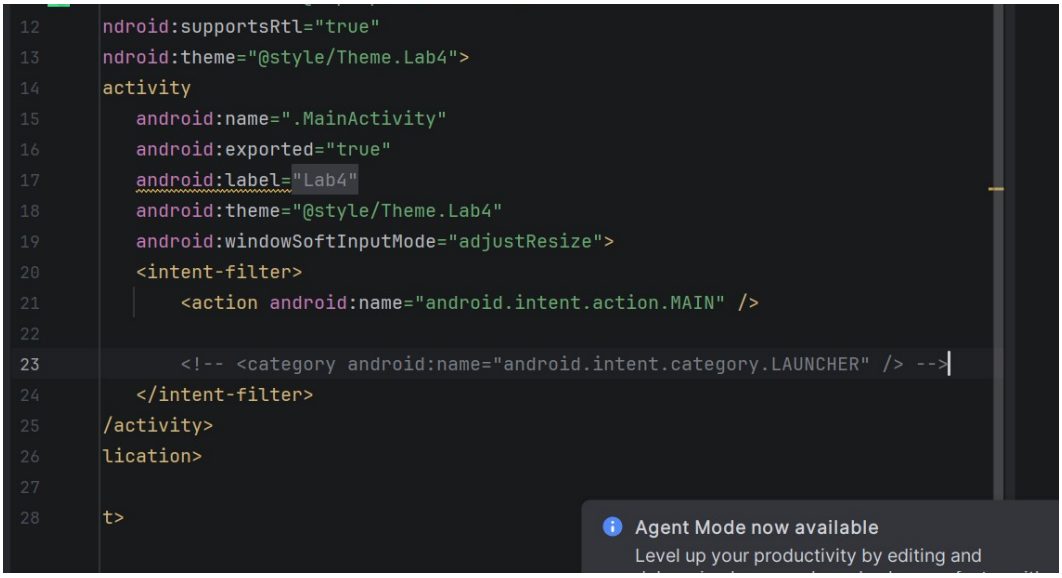


Figure 1. LAUNCHER category temporarily disabled in the manifest.

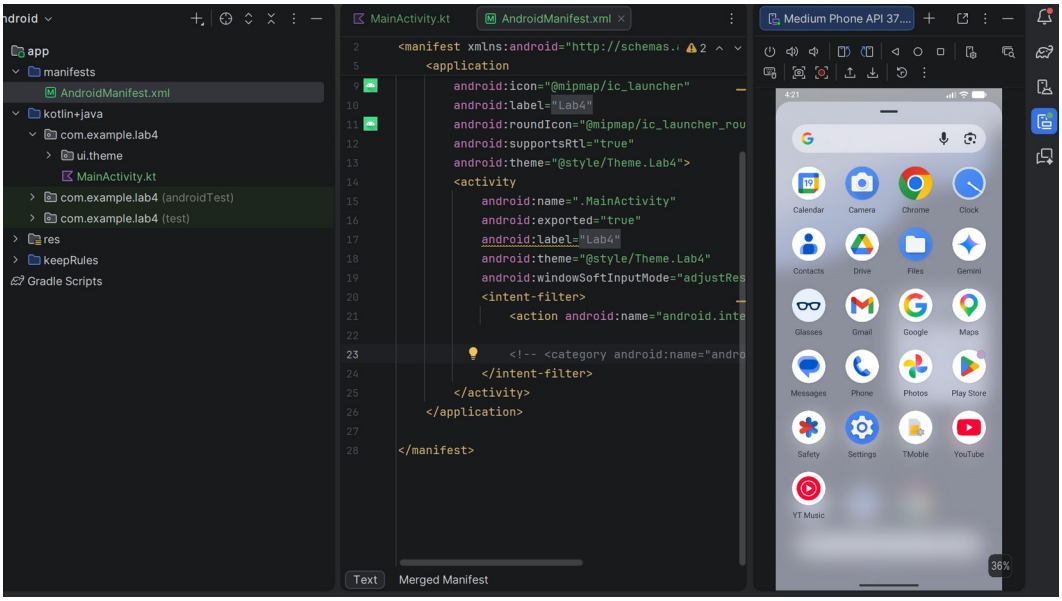


Figure 2. App drawer after disabling the launcher category.

Q1.5: After restoring LAUNCHER, the app could be launched again and displayed Hello Android!.

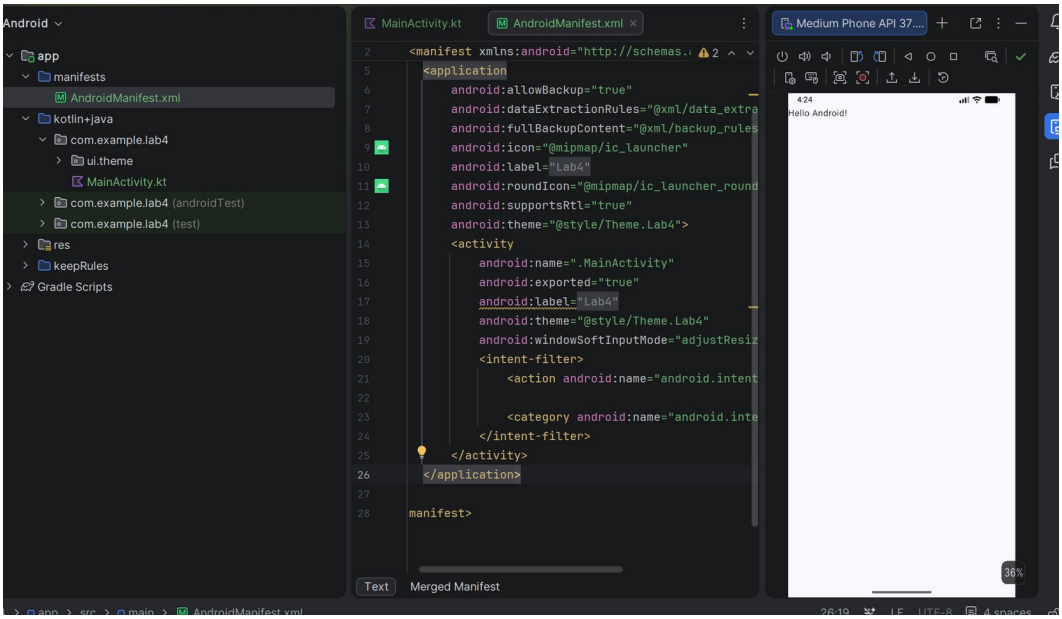


Figure 3. Lab4 launches after the LAUNCHER category is restored.

3. Task 2 — Model the UI State

The Attendance screen displays a session title, a check-in count, and a Check in button. The data class AttendanceUiState contains sessionTitle (String), checkedInCount (Int), and isCheckedIn (Boolean). The user event is onCheckIn.

4. Task 3 — Stateless Composable and Previews

AttendanceScreen accepts uiState and onCheckIn as parameters. It shows the title and count and disables the button when isCheckedIn is true. Two Compose previews represent different UI states: 12 check-ins before the action and 13 after the action.

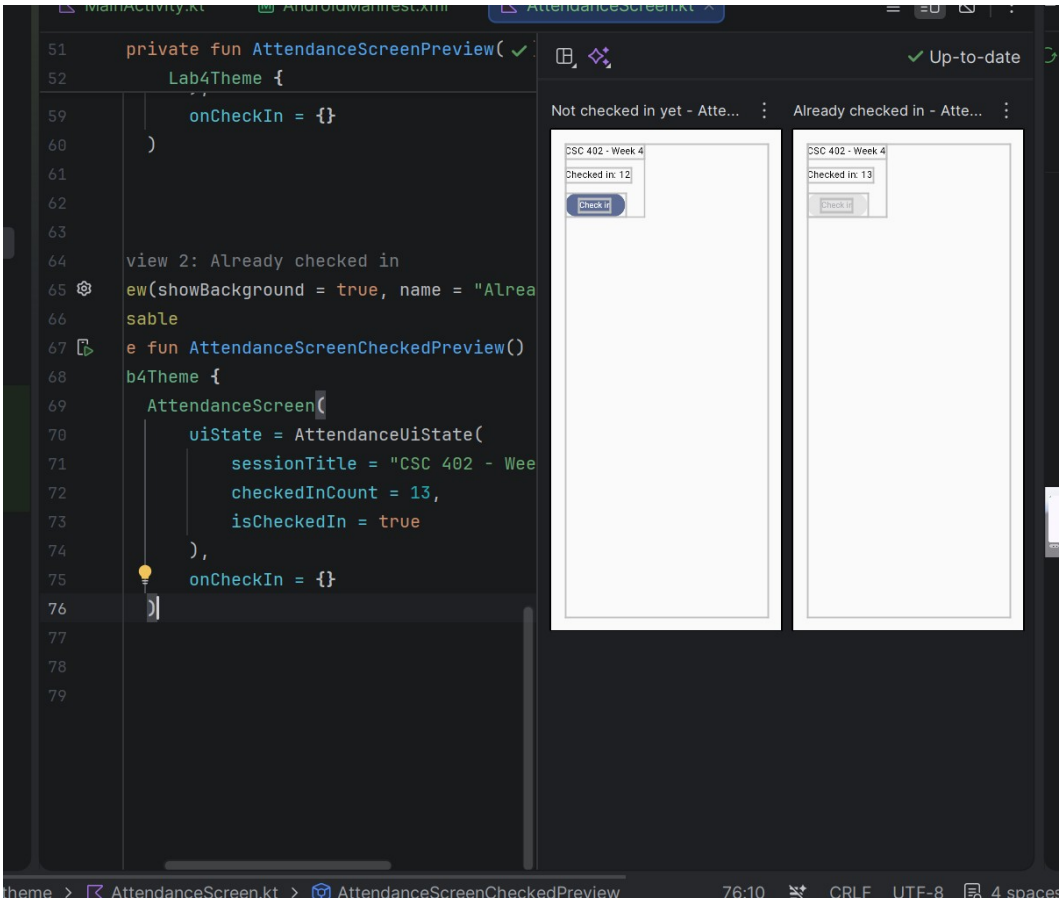


Figure 4. AttendanceScreen previews showing enabled and disabled button states.

5. Task 4 — Route, Events, and State Updates

AttendanceRoute owns a remembered mutable AttendanceUiState. On button press, it creates and assigns a new state using uiState.copy(checkedInCount = uiState.checkedInCount + 1, isCheckedIn = true). Compose recomposes the screen with the updated values.

Observed result: the count changed from 12 to 13 and the Check in button became disabled.

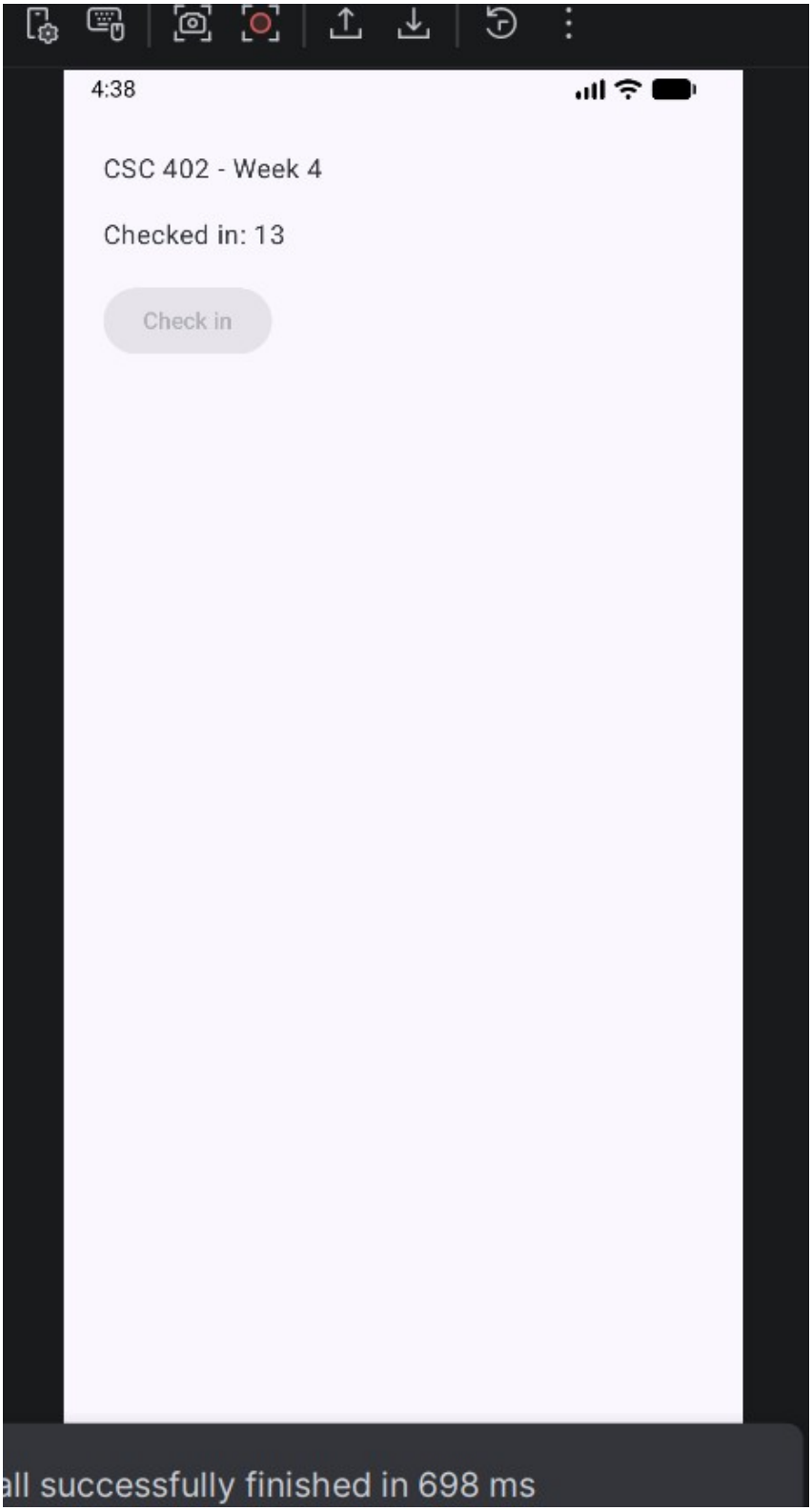


Figure 5. Attendance app after a successful check-in.

Broken update experiment: replacing the callback with uiState.checkedInCount + 1 only computes a value; it does not assign new state. The count therefore remains 12 and the button stays enabled. The correct copy-based handler was restored afterward.

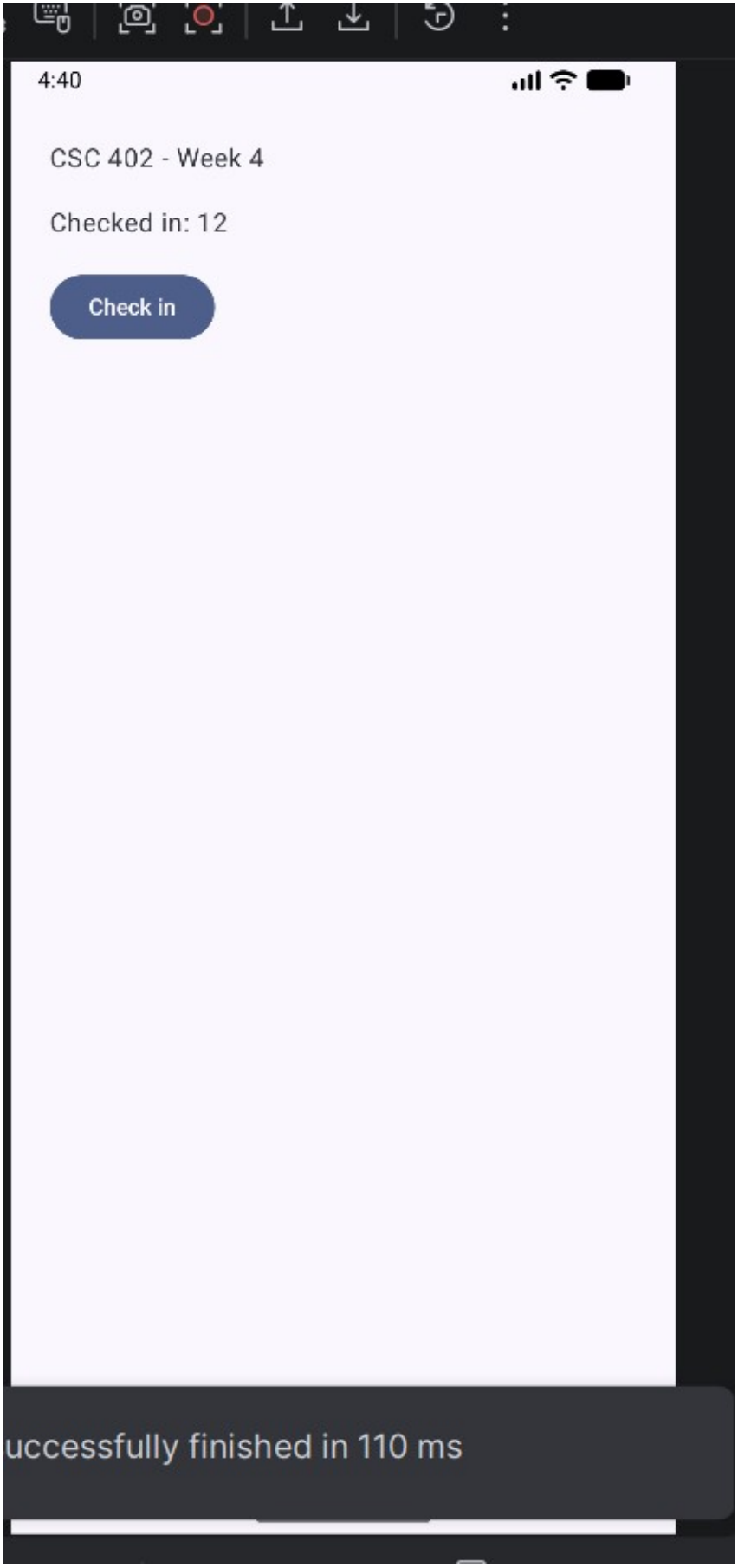

4:40
CSC 402 - Week 4
Checked in: 12
Check in
uccessfully finished in 110 ms

Figure 6. Broken update experiment; the state does not change.

6. Individual Assignment — StudyPlanner (5 Marks)

6.1 Paper Sketch and Screen Plan (P1.1)

The paper sketch outlines the updated StudyPlanner home screen with a greeting, study progress (3/7), a progress bar, total and completed task cards, a Complete a Task button, and a motivation section. Displayed items: greeting, study progress, total tasks, completed tasks, progress bar, and completion button. Mutable state: completedTasks. User interaction: tapping Complete a Task increases the completed count by one and updates the progress bar and completion percentage. Button state: the button is disabled when all seven tasks are complete.

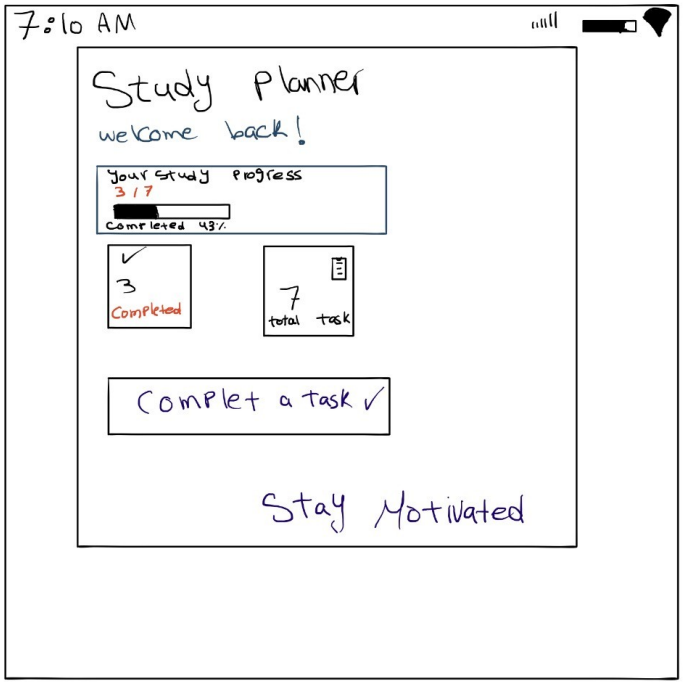


Figure 7. Updated hand-drawn StudyPlanner screen showing the main interface and progress controls.

6.2 Project and UI State (P1.2–P1.3)

A separate Android Studio project named StudyPlanner was created. PlannerUiState is a Kotlin data class with default values: greeting = "Welcome back!", totalTasks = 7, and completedTasks = 3. The implemented package is com.example.studyplanner. The lab handout lists sa.edu.iau.cshj.studyplanner, so the package name should be confirmed against the submission requirements.

6.3 Stateless Home Screen and Previews (P1.4–P1.5)

PlannerHomeScreen(uiState, onCompleteTask, modifier) is stateless and displays the greeting, progress card, percentage, task summary cards, Today's Goal, and a motivation message. It enables Complete a Task only while completedTasks < totalTasks. The two previews demonstrate the in-progress state (3/7) and the completed state (7/7), including the disabled button.

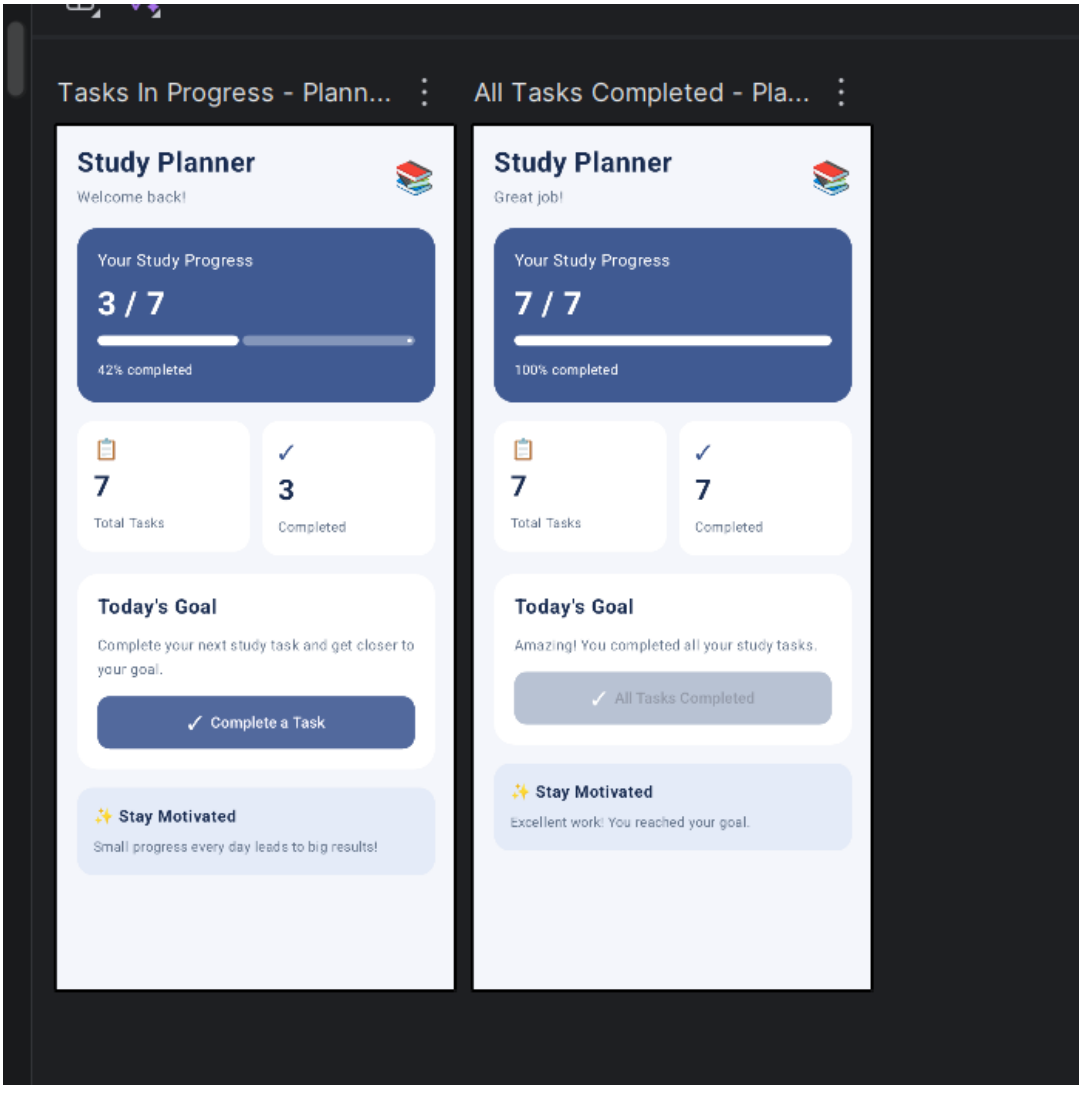


Figure 8. Updated StudyPlanner previews: 3/7 in progress and 7/7 completed.

6.4 Route, Interaction, and Testing (P1.6)

PlannerHomeRoute owns the mutable PlannerUiState using remember and mutableStateOf. The onCompleteTask callback assigns a new state using uiState.copy(completedTasks = uiState.completedTasks + 1). The running app was tested on the emulator: its task count, percentage, and progress bar updated with each tap, including 5/7 (71%). At 7/7 (100%), the button became disabled and the completion message appeared.

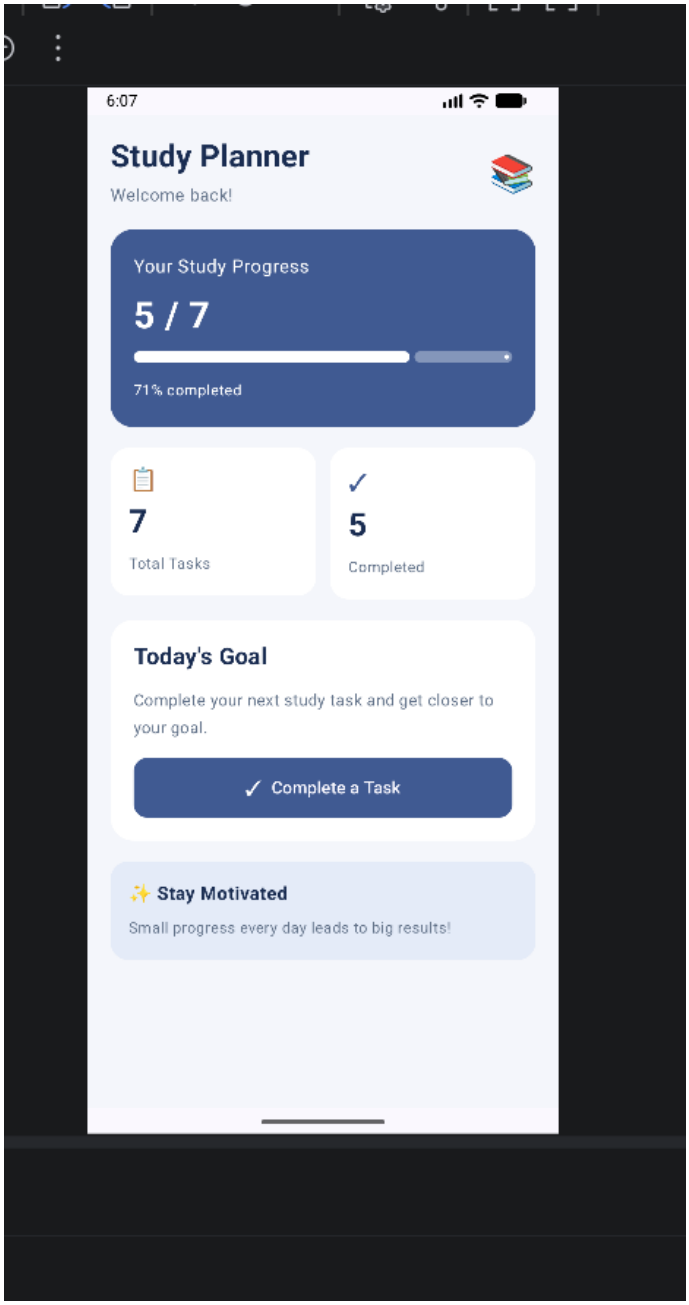


Figure 9. Updated StudyPlanner on the emulator at 5/7 completed (71%).

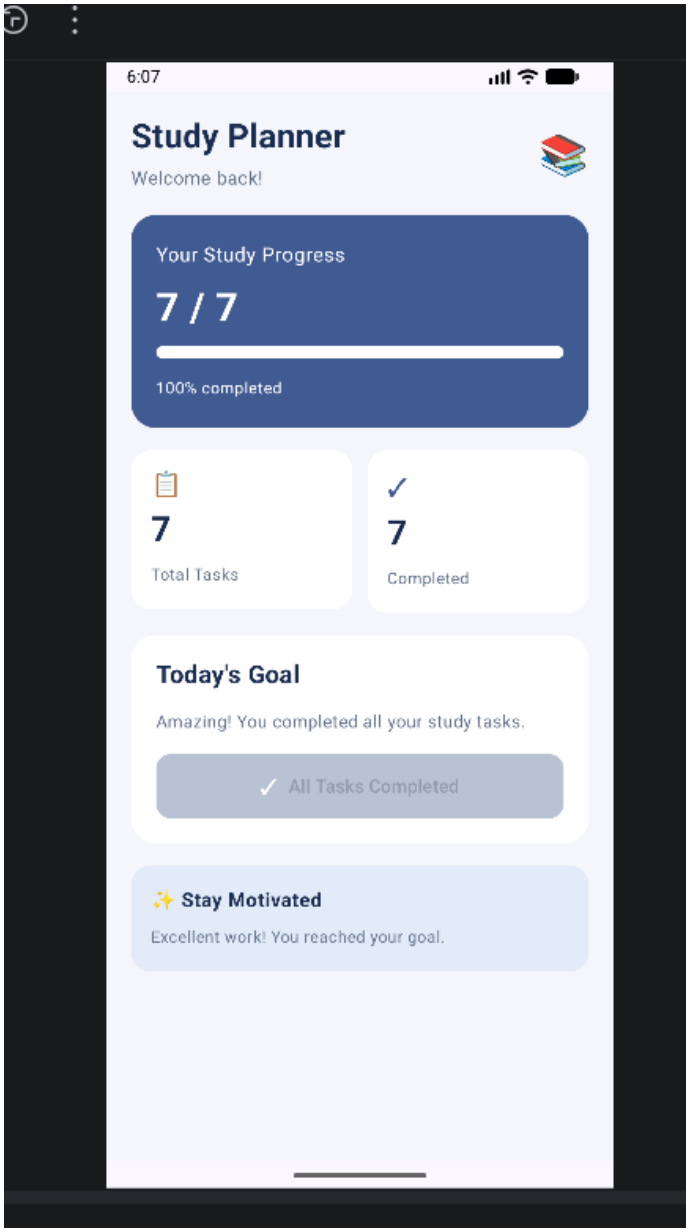


Figure 10. Updated StudyPlanner at 7/7 completed (100%) with the button disabled.

6.5 Manifest Configuration (P1.7)

AndroidManifest.xml declares exactly one Activity, .MainActivity, with android:exported="true". Its intent filter contains android.intent.action.MAIN and android.intent.category.LAUNCHER. The application label uses @string/app_name. The configured label should be checked in strings.xml.

7. Results and Conclusion

Both Compose projects were run and tested. Lab4 demonstrated that launcher configuration controls discoverability and that recomposition depends on assigning updated state. StudyPlanner demonstrated a stateless home screen, two meaningful

previews, a Route that updates state with copy(), and a completion button that becomes disabled at the task limit.

8. AI Assistance Disclosure

AI assistance was used to help troubleshoot Android Studio compilation and theme-reference errors, and to suggest refinements to the StudyPlanner interface and report wording. The application was run and its task-count updates, progress display, previews, and disabled-button behavior were checked on the emulator.